

CS3002E Compiler Design

Monsoon 2025

Lecture #14

Bottom-Up Parsing - 3: LR Parsing, LR(0) items

Department of Computer Science & Engineering
NIT Calicut

Shift-Reduce Parsing: Example

$S \rightarrow L = R$

$L \rightarrow id$

$R \rightarrow num$

Stack	Input	Action
\$ 0	<i>id</i> = <i>num</i> \$	shift
\$ 0 <i>id</i> 1	= <i>num</i> \$	reduce by $L \rightarrow id$
\$ 0 <i>L</i> 2	= <i>num</i> \$	shift
\$ 0 <i>L</i> 2 = 3	<i>num</i> \$	shift
\$ 0 <i>L</i> 2 = 3 <i>num</i> 4	\$	reduce by $R \rightarrow num$
\$ 0 <i>L</i> 2 = 3 <i>R</i> 5	\$	reduce by $S \rightarrow L = R$
\$ 0 <i>S</i> 6	\$	accept

Parser States indicated by items

Stack	Input	Action	State: Items
\$ 0	$id = num\$$	shift	0 : $S \rightarrow .L = R$, $L \rightarrow .id$
\$ 0 id 1	$= num\$$	reduce by $L \rightarrow id$	1 : $L \rightarrow id.$
\$ 0 L 2	$= num\$$	shift	2 : $S \rightarrow L. = R$
\$ 0 L 2 = 3	$num\$$	shift	3 : $S \rightarrow L = .R$, $R \rightarrow .num$
\$ 0 L 2 = 3 num 4	\$	reduce by $R \rightarrow num$	4 : $R \rightarrow num.$
\$ 0 L 2 = 3 R 5	\$	reduce by $S \rightarrow L = R$	5 : $S \rightarrow L = R.$
\$ 0 S 6	\$	accept	

State: Set of items

A parser state is represented by a **set of items**.

- $A \rightarrow \alpha.\beta$ in a state indicates that the parser has already seen a string derivable from α , and is expecting to see a string derivable from β
- $A \rightarrow \alpha\beta.$ in a state indicates that the parser has seen the entire right hand side of production (may lead to a reduction by $A \rightarrow \alpha\beta$)

LR Parsing

- L for left-to-right scanning of the input, R for constructing a rightmost derivation in reverse
- Maintain states to keep track of progress of parsing
- Each state represents a *Set of items*

LR(0) items

- An item of the form $A \rightarrow \alpha.\beta$ is known as an LR(0) item.
- A state of the parser is defined by a set of LR(0) items.

Augmented Grammar

$$S' \rightarrow S$$

$$S \rightarrow L = R$$

$$L \rightarrow id$$

$$R \rightarrow num$$

- A new start symbol S' and a production $S' \rightarrow S$
- New production to indicate acceptance.
 - Parser accepts when it is about to reduce by $S' \rightarrow S$

Closure of Set of Items

For a set of items I , $Closure(I)$ is computed as follows:

- Add every item in I to $Closure(I)$.
- If $A \rightarrow \alpha.B\beta$ is in $Closure(I)$ then for each production $B \rightarrow \gamma$ add $B \rightarrow .\gamma$ to $Closure(I)$.
- Repeat the above step until no more new items can be added to $Closure(I)$.

Closure: Example

$$S' \rightarrow S$$
$$S \rightarrow L = R$$
$$L \rightarrow id$$
$$R \rightarrow num$$
$$Closure(\{S \rightarrow .L = R\}) = \{S \rightarrow .L = R, L \rightarrow .id\}$$
$$Closure(\{S \rightarrow L = .R\}) = \{S \rightarrow L = .R, R \rightarrow .num\}$$

Closure: Example

$$E' \rightarrow E$$
$$E \rightarrow E + T \mid T$$
$$T \rightarrow id$$
$$Closure(\{E \rightarrow .E + T\}) = \{E \rightarrow .E + T, E \rightarrow .T, T \rightarrow .id\}$$

Closure: Example

$$S' \rightarrow S$$
$$S \rightarrow L = R$$
$$L \rightarrow id$$
$$R \rightarrow num$$
$$Closure(\{S' \rightarrow .S\}) = \{S' \rightarrow .S, S \rightarrow .L = R, L \rightarrow .id\}$$

Goto

$Goto(I, X)$ where I is a set of items and X is a grammar symbol, is defined to be the closure of the set of all items $A \rightarrow \alpha X \beta$ such that $A \rightarrow \alpha X \beta$ is in I .

Goto: Example

$$S' \rightarrow S$$
$$S \rightarrow L = R$$
$$L \rightarrow id$$
$$R \rightarrow num$$

$Goto(\{S \rightarrow L. = R\}, =)$
is $\{S \rightarrow L = .R, R \rightarrow .num\}$

Goto: Example

$$E' \rightarrow E$$

$$E \rightarrow E + T \mid T$$

$$T \rightarrow id$$

$$Goto(\{E \rightarrow E. + T\}, +) = \{E \rightarrow E + .T, T \rightarrow .id\}$$

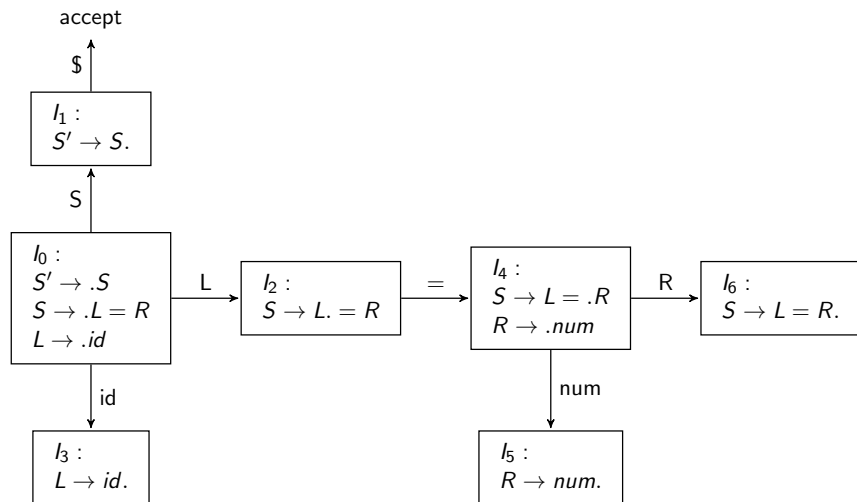
$$Goto(\{E \rightarrow E + .T, T \rightarrow .id\}, T) = \{E \rightarrow E + T.\}$$

LR(0) Automaton

- Each state represented by a set of LR(0) items.
- Initial state is $Closure(\{S' \rightarrow .S\})$
- State transitions given by the $GOTO()$ functions.

Note: The collection of sets of LR(0) items is known as the **Canonical LR(0) collection**.

LR Parser State Transition Diagram



Reference:

- Aho A.V., Lam M.S., Sethi R., and Ullman J.D. Compilers: Principles, Techniques, and Tools (ALSU). Pearson Education, 2007.

Further reading:

- ALSU chapter 4, section 4.6